

HiSUP 2.0: Viva Preparation Guide

This document is designed to help you prepare for your project viva. It details the system architecture, how the frontend and backend communicate, how data is added and processed, and how all database entities are connected.

1. System Overview & Tech Stack

HiSUP 2.0 is a comprehensive University Enterprise Resource Planning (ERP) system.

- **Frontend:** React (built with Vite). Uses `Axios` for API calls and `Context API` for managing authentication state.
- **Backend:** Node.js with Express. Acts as the middle layer, exposing RESTful APIs, handling authentication via JSON Web Tokens (JWT), and managing business logic.
- **Database:** PostgreSQL hosted on **Supabase**. Supabase is used not just for standard CRUD operations, but extensively relies on **PostgreSQL Stored Procedures (RPCs)** for transactional safety.

2. System Architecture & Flow

When the panel asks *how a request travels through your system*, explain this flow:

1. **User Interaction:** The user interacts with the React frontend (e.g., clicks "Enroll" on a course).
2. **API Request:** `Axios` sends an HTTP request to the Express Backend, attaching a JWT token in the `Authorization` header.
3. **Backend Middleware:** The backend intercepts the request. The `auth.middleware.js` verifies the JWT to ensure the user is logged in and checks if they have the correct role (e.g., Student vs. Admin).
4. **Controller & Database:** The controller handles the request. Instead of writing raw SQL, it uses the `@supabase/supabase-js` client to execute operations. For complex logic, it calls a Supabase RPC (Remote Procedure Call).
5. **Response:** The database updates, the backend responds with success/failure JSON, and the frontend updates the UI.

3. Database Schema & Connections (How everything is connected)

The database is highly relational. Here is how the major components are tied together:

Core Users & Roles

- **user_accounts:** The central table for all users. It holds `username`, `email`, `password_hash`, and a `role` (Admin, Student, Faculty, Finance).
- **students** and **faculty:** These tables extend `user_accounts`. A student record links back to `user_accounts` via `user_id`.

Academic Structure

- **departments:** The top level (e.g., Computer Science).
- **programs:** Belongs to a department (e.g., BSCS, MSCS). Connected via `department_id`.
- **courses:** Belongs to a department. Contains the course `syllabus` definition.
- **sections:** A specific offering of a course in a specific semester. It links `course_id` and `faculty_id` (who is teaching it) and tracks `capacity` and `available_seats`.

The Core Activity: Enrollments & Attendance

- **enrollments:** A junction table that connects a `student_id` to a `section_id`.
- **attendance_records:** Tracks attendance. Connects `student_id`, `section_id`, a specific date, and a status (Present/Absent).
- **grades & results:** Links students to sections to record final grades, and aggregates this into semester GPAs.

Other Modules

- **Finance:** `fee_structures` defines fees per program. `student_fees` tracks what a student owes. `fee_payments` records their payment attempts.
- **Library:** `library_items` is the inventory. `library_issues` connects a `book_id` to a `user_id` with a `due_date`.
- **Hostel:** `hostels` contain rooms. `hostel_allotments` links a `student_id` to a `room_id`.

4. How Data is Added (The "Insertion" Process)

The panel will likely ask: *"How do you handle complex insertions like enrollment or payments?"*

The answer is: **PostgreSQL Stored Procedures (Transactions)**.

We use stored procedures in the database to guarantee that if one step fails, the whole process rolls back (ACID compliance). Here are the key mechanisms:

Example 1: Enrolling in a Course (`enrollstudentincourse`)

When a student enrolls, several things must happen simultaneously. The Express backend calls `supabase.rpc('enrollstudentincourse', ...)` which executes the following SQL logic:

1. **Validation:** Checks if the student is 'Active'.
2. **Finance Check:** Calls a function `fn_outstanding_fee` to ensure the student has no pending fees.
3. **Seat Availability Check:** Locks the row in the `sections` table and checks if `available_seats > 0`.
4. **Insertion:** Inserts a new row into `enrollments`.
5. **Update:** Decrements `available_seats` by 1 in the `sections` table.
6. **Audit & Notify:** Logs the action in `audit_logs` and sends a notification to the student.
(If any step fails, the whole transaction is aborted).

Example 2: Approving a Fee Payment (`approvepayment`)

When Finance approves a payment:

1. It updates the `status` in `fee_payments` to 'Verified'.
2. It updates the student's overall ledger in `student_fees`, adding to paid and deducting from remaining.
3. It generates a receipt in the `receipts` table.
4. It logs the audit trail and notifies the student.

Example 3: Issuing a Library Book (`issuebook`)

1. Checks if `available_copies > 0` in `library_items`.
2. Inserts a record into `library_issues` with a `due_date` (14 days from now).
3. Decrements `available_copies` in `library_items`.

Example 4: Room Allocation (`allocate room`)

1. Checks if `occupied < capacity` for a specific room.
2. Checks if the student doesn't already have an active room.
3. Inserts into `hostel_allotments`.
4. Increments `occupied` count in the `rooms` table.

5. Security & Access Control

- **Authentication:** JWT (JSON Web Tokens). When a user logs in, the backend encrypts their ID and Role into a token. The frontend saves this in `localStorage` and sends it with every request.
- **Authorization (Backend):** The `auth.middleware.js` intercepts requests. If an endpoint is meant for Admins (like adding a department), the middleware checks `req.user.role === 'Admin'` and rejects it otherwise.
- **Frontend Protection:** React components check the `AuthContext`. If a student tries to access the Finance page, the router redirects them.

6. Common Viva Questions to Anticipate

1. **Why use Supabase instead of direct PostgreSQL?**
Answer: Supabase provides a managed PostgreSQL database, an excellent SDK for querying, and built-in connection pooling, which speeds up development while keeping the power of raw Postgres (RPCs, Triggers).
2. **How did you prevent two students from enrolling in the last seat at the same time?**
Answer: We used SQL row-level locking (`FOR UPDATE`) inside our stored procedure (`enrollstudentincourse`). It locks the section row until the transaction finishes.
3. **How is the password secured?**
Answer: We don't store plain text. We hash the password using `bcryptjs` before storing it in `user_accounts`.
4. **What happens if a student drops out?**
Answer: We update their `status` to 'Dropped' or 'Suspended' in the `students` table. Because of Foreign Key constraints (`ON DELETE CASCADE` or `RESTRICT`), their past data remains intact, but our stored procedures prevent them from making new enrollments or issuing books.